

Debugging with ipdb (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-debugging-with-ipdb>  <https://github.com/learn-co-curriculum/python-p3-debugging-with-ipdb/issues/new>

Learning Goals

- Explain how `ipdb` is a more flexible REPL than the Python shell.
- Install `ipdb` via Pipfile.
- Debug a program using `ipdb.set_trace()` within the body of your file.

Key Vocab

- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **REPL (Read-Eval-Print Loop):** A REPL (Read-Eval-Print Loop) is a special tool for programming where you can type in code and see what happens right away. It's like having a conversation with the computer. You say something (type in code), the computer understands it and does what you asked, and then it shows you the result.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

We'll cover `ipdb`, a type of REPL, and discuss how to install and use it to debug a program.

What Is a REPL?

You've already been introduced to REPLs through using the Python Shell. REPL stands for *Read, Evaluate, Print, Loop*. It is an interactive programming environment that takes a user's input, evaluates it and returns the result to the user.

Python installs with its own REPL, which is the Python shell that you've already been using. Every time you type `python` into your terminal, you're entering into a REPL.

What Is `ipdb` ?

`ipdb` is another Python REPL with some added functionality. It is built on top of `pdb`, a REPL in Python's standard library, and provides helpful features such as tab completion, syntax highlighting, and better tracebacks. When you enter `ipdb`, you are entering a brand new interactive environment. For any code you want to play with in the Python shell, you have to copy and paste or write your code in the Python shell itself. `ipdb`, on the other hand, is like a REPL that you can inject into your program.

`ipdb` is far more flexible than the Python shell. Once you install the `ipdb` library (via this lesson's `Pipfile`), you can use `ipdb.set_trace()` anywhere in your code.

`ipdb.set_trace()` gives you similar functionality to using `debugger` in a JavaScript application, in that it lets you set a **breakpoint** in your code that will pause the execution of your program at a certain point so you can inspect the variables, functions, and other context available at a specific place in your code.

So when you place the line `ipdb.set_trace()` in your code, that line will get interpreted at runtime (as your program is executed). When the interpreter hits that line, your program will actually *freeze* and your terminal will turn into a REPL that exists right in the middle of your program, wherever you added the `ipdb.set_trace()` line.

Let's take a look. In this repository, you'll see a file called `ipdb_is_awesome.py`.

Using `ipdb.set_trace()`

First, let's make sure we have installed the packages from our `Pipfile`, including `ipdb`.

```
> pipenv install
> pipenv shell
```

NOTE: This will only install `ipdb` in your **virtual environment**. If you want to install `ipdb` system-wide, you must exit your virtual environment and run the following command in your local environment:

```
> pip install ipdb
```

Look at the code in `lib/ipdb_is_awesome.py`. You should see the following code:

```
import ipdb

def tracing_the_function():
    inside_the_function = "We're inside the function"
    print(inside_the_function)
    print("We're about to stop because of ipdb!")
    ipdb.set_trace()
    this_variable_hasnt_been_interpreted_yet = \
        "The program froze before it could read me!"
    print(this_variable_hasnt_been_interpreted_yet)

tracing_the_function()
```

At the top of the file, `import ipdb` is a statement that loads the `ipdb` library in this file when our application runs, very similar to `import` statements in JavaScript.

We will elaborate on Python `import` statements later in Phase 3, but note now that Python `import` statements **do not** use quotes or semicolons.

After importing `ipdb`, *which you must do to use `ipdb`*, we define a function, and then call that function.

In the directory of this repo, in your terminal, run the file by typing `python lib/ipdb_is_awesome.py`. Now, look at your terminal. You should see something like this:

```
> python lib/ipdb_is_awesome.py
We're inside the function
We're about to stop because of ipdb!
> /lib/ipdb_is_awesome.py(11)tracing_the_function()
   10      this_variable_hasnt_been_interpreted_yet = \
---> 11          "The program froze before it could read me!"
   12      print(this_variable_hasnt_been_interpreted_yet)
```

```
ipdb>
```

You have frozen your program *as it executes* and are now inside a REPL *inside your program*. You basically just stopped time! How cool is that?

In the terminal, in your `ipdb` console, type the variable name `inside_the_function` and hit enter. You should see a return value of `"We're inside the function"`.

```
ipdb> inside_the_function
"We're inside the function!"
```

You are able to explore the data *inside* the function in which you've placed your binding. Any variables or functions that are available from the line you placed your `ipdb.set_trace()` will be available to you to explore from the terminal.

NOTE: Any names that have been assigned to variables (or functions, methods classes, etc.) can be auto-completed by hitting `tab`. If your name doesn't autocomplete, it hasn't been defined yet!

Now, in the terminal, in your `ipdb` console, type the variable name `this_variable_hasnt_been_interpreted_yet` . You should see a `NameError` . That's because the binding you placed on line 7 actually froze the program on line 7 and the variable you just called hasn't been interpreted yet. Consequently, our REPL doesn't know about it.

Now, in the terminal, type `c` (for **continue**), and you'll leave your `ipdb` console and the program will continue to execute.

Using `ipdb` to Debug

In addition to *exploring* code inside `ipdb` , you can also manipulate variables and try code out. This is where `ipdb` really becomes helpful for debugging. If you have a function that isn't doing what it's supposed to do, instead of making changes in your text editor and running the tests over and over until you get it working, you can put a binding in your code and try things out. Once you've figured out how to fix the problem, you then update the code in your text editor accordingly.

Let's walk through an example together. In this repository, you'll see a `testing` folder containing a file `ipdb_debugging_test.py` . This is a test for the file `lib/ipdb_debugging.py` .

In `ipdb_debugging.py` , we have a broken function. Run `pytest` to see the failing test. You should see the following:

```
=== FAILURES ===
___ TestIpdbDebugging.test_adds_two ___

self = <testing.ipdb_debugging_test.TestIpdbDebugging object at 0x105862fa0>

    def test_adds_two(self):
        '''adds_two() adds 2 to input arg and returns sum'''
>       assert(plus_two(3) == 5)
E       assert 3 == 5
E       + where 3 = plus_two(3)

testing/ipdb_debugging_test.py:10: AssertionError
=== short test summary info ===
```

```
FAILED ipdb_debugging.py adds_two() adds 2 to input arg and returns sum - assert 3 == 5
=== 1 failed in 0.04s ===
```

So what's happening? In our **FAILURES** section, we can see that the test is calling the **plus_two** function and passing in **3** as an argument. Below that we can see that the test is expecting **5** to be returned, but that **3** is being returned instead. Let's take another look at the code in **ipdb_debugging.py**:

```
def plus_two(num):
    num + 2
    return num
```

So while our function is adding 2 to **num** in the first line of our function, it appears that it is not *updating* **num**. We have imported **ipdb** at the top of our **testing/ipdb_debugging_test.py** file so we can use it to verify this. Let's place an **ipdb.set_trace()** in our code, right after that line:

```
def plus_two(num):
    num + 2
    ipdb.set_trace()
    return num
```

Now, run **pytest -s**  (<https://docs.pytest.org/en/latest/how-to/capture-stdout-stderr.html#setting-capturing-methods-or-disabling-capturing>) (the **-s** allows us to see and interact with our code's output during testing) and drop into your **ipdb** console. Your terminal should look like this:

```
ipdb_debugging.py adds_two() adds 2 to input arg and returns sum > /lib/ipdb_debugging.py(8)plus_two()
      6      num + 2
      7      ipdb.set_trace()
----> 8      return num

ipdb>
```

Let's check our current return value by typing **num** at the **ipdb** prompt. You should see something like this:

```
ipdb> num  
3  
ipdb>
```

By checking the value of the variable inside our ipdb console, we can confirm that `num` is still equal to `3` and, as a result, the function is returning `3`.

How can we modify the code so that the function behaves in the expected way? We need to *update* the value of our `num` variable so that it's equal to the sum of itself and 2. Play around inside your `ipdb` console: try code that you think will update `num` as needed, then check the value of `num` to see if it worked. Once you figure it out you can type `exit` in your terminal to get out of `ipdb`, update the code in your text editor, and rerun the test to verify it's passing. Be sure to remove the `ipdb.set_trace()` !

It can take a little while to get the hang of using `ipdb`, so don't worry if it's still a little confusing. As you start working with more complex functions and data structures, you'll find it can be a very helpful tool.

Resources

- [ipdb documentation](https://pypi.org/project/ipdb/) ➞ [\(https://pypi.org/project/ipdb/\)](https://pypi.org/project/ipdb/)
- [Python Debugging with PDB](https://realpython.com/python-debugging-pdb/) ➞ [\(https://realpython.com/python-debugging-pdb/\)](https://realpython.com/python-debugging-pdb/)
- [Capturing output with pytest](https://docs.pytest.org/en/latest/how-to/capture-stdout-stderr.html#setting-capturing-methods-or-disabling-capturing) ➞ [\(https://docs.pytest.org/en/latest/how-to/capture-stdout-stderr.html#setting-capturing-methods-or-disabling-capturing\)](https://docs.pytest.org/en/latest/how-to/capture-stdout-stderr.html#setting-capturing-methods-or-disabling-capturing)

This tool needs to be loaded in a new browser window

Load Debugging with ipdb (CodeGrade) in a new window